

State of the Art in Technology: AI Lecture Notes, Day 2

Main point of this session:

Describe the mathematical ideas behind machine learning and neural networks.

Learning objectives for today:

- Play with gradient descent.
 - Articulate the steps in a neural network.
 - Formulate a metric for a model's success.
-

Day 2 is all about seeing how AI works.

[Elusive](#), by Laker Newhouse and GPT-2 (as students walk in) [Play song on repeat.]

Welcome, everybody! We'll have a song from GPT-2 each morning. This is a song I call "Elusive." You'll notice pictures in the background generated by stable diffusion in the style of watercolor. See those lightning bolts? They look a tiny bit like neural networks to me...

Review: Pipeline of AI (10 minutes)

What are the four steps in the pipeline for AI? Data, model, training, inference.

What do those four steps look like in the example of linear regression?

Three big questions: How to get data? How to define a model? How to train the model? We'll answer these questions in reverse order, starting with where we are more sure of the answers. To do so, let's start with a game!

Ground Rules (2 minute)

Ground rules before we begin:

- 1) Today we'll start on some math. If you don't understand something I'm saying, please stop me. I don't want to leave anyone behind.
- 2) We'll have more interactive demos today. After the demo finishes, I'll tell you to close your laptop screens.

Activity: Gradient Descent Game (10 minutes)

On a whiteboard, draw out a graph of a function. Except, draw only one point. The game is for the students to find the lowest point on the function. The way they can get information is by querying me one x-value at a time, and I'll tell them the y-value. The goal is to find the global minimum in the smallest number of guesses.

And by the way, we are not going to minimize just some random function. By the end of today, we are going to minimize a function that tells us how well our neural network is doing at its task. More on that later. For now, who has ideas for what point to sample next?

[Thinking outside of the box is good. Newton's method (draw line until it intersects the x-axis) or momentum gradient descent are also good ideas.]

Direct students toward using local curvature information to take small steps.

Key takeaways:

1. Looking at nearby points tells you the slope of where you're standing.
2. Taking a proper sized step (not too big, not too small) gets you downhill fastest.

Introduction to Derivatives (10 minutes)

How can we quantify that local information about slope? We want nearby points, but how nearby? The derivative is our tool for finding the slope. Demonstrate point getting closer and closer on the whiteboard. No limit definition. [Show [demo on Desmos](#).]

Introduction to Gradient Descent (10 minutes)

Next, how can we hike downhill in 2D (on a mountain)? Use a student's jacket or piece of cardboard to make a curved surface. Hold it up. Ask the students how they would find the direction to walk. Interactive! Take suggestions!

Open laptops and go to [interactive demo](#).

Takeaway: we can decompose 2D slope into two 1D derivatives.

Close laptops. State that we can extend into n-D. Draw list $(x_1, x_2, \dots, x_n)$ on whiteboard. Demonstrate that for each variable, we can test whether increasing it makes

us go uphill or downhill — whether our neural network does better or worse. We check every variable, then all at once we take a small step in the right direction for each one. This process is called gradient descent.

Activity: Momentum Gradient Descent (5 minutes)

Momentum gradient descent [interactive demo](#).

Key point: we can be creative about how we navigate down the mountain.

Introduction To Neural Networks (20 minute)

Let's begin now to build up to a function that we will want to minimize. We're stepping now from "how to train a model" to "how to define a model." That shift means we're now going to build a massive function. It will have not just two knobs (like the line from before) but thousands of knobs to turn. Once we understand that function, gradient descent will do the magic work of finding inputs that make the function output a small number. I'm excited to say it: let's define a neural network. Ask questions at any point.

Example: neural network with neuron shape (2, 3, 1). Input = (years worked, years in school), output = (salary per year in thousands).

1. Draw the layers
2. Explain each neuron holds a number
3. Explain each neuron connects to all the next neurons
4. Add up weights * activations, then apply nonlinearity (explain nonlinearity last)
5. Input data in first layer, output answer in last layer

Why do we need a nonlinearity? For those of you who were at the linear algebra watch party last night, see if you can connect what I'm about to say to the language of linear transformations. And if you weren't there, that's fine. Imagine that we would only have neurons that added and scaled numbers. No matter how big or complicated we made our neural network, it would always just be adding and multiplying. There's not enough complexity here to capture intelligence.

Two examples of nonlinearities: sigmoid and ReLU. Draw on the whiteboard.

— BREAK (5 minutes) —

Neural Networks: MNIST (15 minutes)

Welcome back! We've just covered what a neural network is, and now I'm going to show it to you again in a new way. Seeing something in two ways, from two perspectives, can help it to stick in your brain more deeply. And what we are about to learn is very important to understand deeply. Neural networks are humanity's most fundamental invention in machine learning. Everything else is based on them.

Watch 3Blue1Brown's [neural network video](#)

Pause at 5:30 — what have you learned so far?

Pause at 11:38 — what have you learned so far?

Stop at 12:41 — any questions, thoughts, or comments? Get students talking! Ask them to summarize what they've seen. Ask them to walk through, aloud, how numbers describe pixels, then turn into numbers in the input layer, then turn into numbers in the hidden layer via weights, then are squished with a nonlinearity, then turn into numbers in the output layer via weights, then are squished with a nonlinearity.

Introduction to Loss Functions (15 minutes)

Finally we have a complex model: a neural network. This model is rich enough to describe a wide variety of tasks. But how do we know if this neural network is doing a good job? After all, I need a way to look at your neural network and tell you how well it is doing; that's how we will run gradient descent later.

What we need here is something called a loss function. The idea is to quantify how “badly” the neural network is doing. In fact, we saw this already in the linear regression example. There, the loss was all the areas of red boxes. Here, let's take a look at examples of what a loss function might look like.

For all example, we need supervised training pairs (data, label). Examples: labels of (years studied, years worked) to (salary), or (pixels in image) to (dog, cat), or (pixels in image) to (0, 1, 2, 3, 4, 5, 6, 7, 8, 9).

Let's take the example of salary. Explain squared loss (for numerical output).

Let's take the example of predicting dog or cat. Here, the number the neural network outputs is not a probability. Quick aside: there's a way we can transform the output to always be a probability distribution. It's called softmax, and it's not particularly important

for now. Suppose the predicted probabilities are dog = 0.8, cat = 0.2. Let's say the correct answer is dog. How well is the model doing? Certainly it would have been better if the model said dog = 1.0, cat = 0.0. How can we quantify this?

Have you ever seen a graph of the log function? Let's take a look. [[Pull up Desmos.](#)]

Hmm... and what if we make the log be negative? Woah, this is perfect. This is just what we want. Let's say we use the log for whatever we want the correct probability to be. If the model puts probability 1, the loss is 0: no error. If the model puts probability close to 0, the loss shoots up very quickly. This gives the model a super strong incentive to make the correct prediction.

Negative log loss is a tool for telling the model how close it comes to the right answer when its output is a probability. What we do is look at whichever one should have been the right answer, and the loss is the negative log of whatever probability the model outputted.

Conclusion (3 minutes)

Today we saw the math behind AI. Key takeaways:

1. Gradient descent is like hiking downhill: it lets us find low points on functions.
2. A neural network has layers of neurons that each activate based on the previous layer. Each neuron applies a nonlinearity so the network can capture enough complexity.
3. Loss functions let us quantify how well a model is doing. Then we can use the loss function in gradient descent.

Today, we learned what a neural network is and how it works. Tomorrow, you will make one in class. Not only that, but we will have an AI lab after the afternoon session tomorrow. I'm calling it the "Transformers Lab." There, you will learn about the neural networks behind ChatGPT. See you tomorrow!